Name: Soham Chavan

PRN: 124B1F022

Class: IT A

Assignment No. 7

Aim:

To design an efficient exam timetable scheduling system using Graph Colouring, where courses are vertices and an edge connects two courses that share at least one student, so that no two conflicting exams are held in the same time slot while the number of slots is minimized. The Greedy, Welsh-Powell and DSATUR algorithms are compared, and exams are assigned to rooms according to capacity.

Theory:

1. Graph Theory

Graph theory is used to model pairwise relationships between objects. A graph G(V, E) consists of vertices (entities such as courses) and edges (relationships such as shared students between courses). Graphs can be directed or undirected, and weighted or unweighted. In this problem the graph is undirected and unweighted, because a conflict between two courses is mutual and has no weight.

2. Graph Colouring

Graph colouring assigns colours to the vertices of a graph so that no two adjacent vertices have the same colour. In timetable scheduling each course is a vertex, an edge connects two courses if at least one student is enrolled in both, and each colour represents a distinct exam time slot. The goal is to minimize the number of colours. The minimum number of colours needed is called the Chromatic Number.

3. Conflict Graph (Problem Modeling)

- Courses with common students are connected by edges.
- If two courses share an edge, they cannot have the same exam slot.
- Hence timetable scheduling becomes a Graph Colouring Problem, and no student has two exams at the same time.

4. Graph Colouring Algorithms

- Greedy Colouring: colours one vertex at a time using the smallest colour not used by its neighbours. Simple, but not always minimum. Time complexity: O(V^2).
- DSATUR (Degree of Saturation): picks the next vertex based on the number of different colours used by its neighbours. Gives near-optimal solutions. Time complexity: O(V^2).
- Welsh-Powell: sorts vertices in descending order of degree and then applies the greedy method. Usually better than simple greedy. Time complexity: O(V^2).

5. Room Allocation Constraint

After time slots (colours) are assigned, the exams in the same slot must be assigned to classrooms based on: (a) room capacity, each exam must fit in its room; (b) availability, a room can host only one exam per

slot; and (c) optimization, the smallest available room that fits all students is chosen to maximize space utilization.

6. Optimization for Large Datasets

- Use adjacency lists instead of matrices to save memory.
- Use hash maps or dictionaries to store student-course relationships.
- Apply sorting and efficient traversal to reduce complexity.
- Build the conflict graph in parallel for faster processing.

Algorithm

1. Build the conflict graph: for every student, add an edge between each pair of courses the student has enrolled in.
2. Initialize the colour of every course to -1 (not assigned).
3. Sort the courses in descending order of degree and colour each course with the smallest colour not used by its adjacent courses.
4. For each time slot, sort the exams by number of students and assign each to the smallest free room that has enough capacity.
5. Display the final timetable (Course, Time Slot, Room).

Code:

```
import java.util.*;

public class ExamTimetable {

    static List<Set<Integer>> adj;
    static int[] size;
    static int V;

    static void buildGraph(int numCourses, List<int[]> enrollments) {
        V = numCourses;
        adj = new ArrayList<>();
        size = new int[V];
        for (int i = 0; i < V; i++) adj.add(new HashSet<>());
        for (int[] courses : enrollments) {
            for (int c : courses) size[c]++;
            for (int a = 0; a < courses.length; a++)
                for (int b = a + 1; b < courses.length; b++) {
                    adj.get(courses[a]).add(courses[b]);
                    adj.get(courses[b]).add(courses[a]);
                }
        }
    }

    static int smallestFreeColour(int u, int[] colour) {
        boolean[] used = new boolean[V + 1];
        for (int v : adj.get(u)) if (colour[v] != -1) used[colour[v]] = true;
        int c = 0;
        while (used[c]) c++;
        return c;
    }
```

```
    // 1. Greedy colouring in the given (natural) order
    static int[] greedy() {
        int[] colour = new int[V];
        Arrays.fill(colour, -1);
        for (int u = 0; u < V; u++) colour[u] = smallestFreeColour(u, colour);
        return colour;
    }

    // 2. Welsh-Powell: sort by degree (descending), then greedy
    static int[] welshPowell() {
        Integer[] order = new Integer[V];
        for (int i = 0; i < V; i++) order[i] = i;
        Arrays.sort(order, (a, b) -> adj.get(b).size() - adj.get(a).size());
        int[] colour = new int[V];
        Arrays.fill(colour, -1);
        for (int u : order) colour[u] = smallestFreeColour(u, colour);
        return colour;
    }

    // 3. DSATUR: pick the vertex with the most distinct neighbour colours (ties ->
higher degree)
    static int[] dsatur() {
        int[] colour = new int[V];
        Arrays.fill(colour, -1);
        for (int step = 0; step < V; step++) {
            int best = -1, bestSat = -1, bestDeg = -1;
            for (int u = 0; u < V; u++) {
                if (colour[u] != -1) continue;
                Set<Integer> seen = new HashSet<>();
                for (int v : adj.get(u)) if (colour[v] != -1) seen.add(colour[v]);
                int sat = seen.size(), deg = adj.get(u).size();
                if (sat > bestSat || (sat == bestSat && deg > bestDeg)) {
                    best = u; bestSat = sat; bestDeg = deg;
                }
            }
            colour[best] = smallestFreeColour(best, colour);
        }
        return colour;
    }

    static int countColours(int[] colour) {
        int m = 0;
        for (int c : colour) m = Math.max(m, c + 1);
        return m;
    }

    static boolean isValid(int[] colour) {
        for (int u = 0; u < V; u++)
            for (int v : adj.get(u)) if (colour[u] == colour[v]) return false;
        return true;
    }

    static void allocateRooms(int[] colour, String[] roomNames, int[] roomCap) {
```

```
        int slots = countColours(colour);
        System.out.printf("%-8s %-10s %-10s %-8s%n", "Course", "Time Slot",
"Students", "Room");
        for (int t = 0; t < slots; t++) {
            List<Integer> exams = new ArrayList<>();
            for (int c = 0; c < V; c++) if (colour[c] == t) exams.add(c);
            exams.sort((a, b) -> size[b] - size[a]);
            boolean[] roomBusy = new boolean[roomNames.length];
            for (int e : exams) {
                int pick = -1;
                for (int r = 0; r < roomNames.length; r++) {
                    if (!roomBusy[r] && roomCap[r] >= size[e] && (pick == -1 ||
roomCap[r] < roomCap[pick])) pick = r;
                }
                if (pick != -1) roomBusy[pick] = true;
                System.out.printf("%-8s %-10s %-10d %-8s%n", "C" + (e + 1), "Slot " +
(t + 1), size[e],
                        pick == -1 ? "NO ROOM" : roomNames[pick]);
            }
        }
    }

    static void runAll(String title, int numCourses, List<int[]> enrollments, String[]
rn, int[] rc) {
        System.out.println("=== " + title + " ===");
        buildGraph(numCourses, enrollments);
        int edges = 0;
        for (Set<Integer> s : adj) edges += s.size();
        System.out.println("Courses (vertices): " + V + ", Conflicts (edges): " +
edges / 2);
        for (int u = 0; u < V; u++) {
            StringBuilder sb = new StringBuilder();
            for (int v : new TreeSet<>(adj.get(u))) sb.append(sb.length() > 0 ? ", " :
"").append("C").append(v + 1);
            System.out.println("  C" + (u + 1) + " (" + size[u] + " students)
conflicts with: " + (sb.length() == 0 ? "none" : sb));
        }
        int[] g = greedy(), w = welshPowell(), d = dsatur();
        System.out.println("\nAlgorithm comparison (number of exam slots):");
        System.out.println("  Greedy        : " + countColours(g) + "  valid = " +
isValid(g));
        System.out.println("  Welsh-Powell  : " + countColours(w) + "  valid = " +
isValid(w));
        System.out.println("  DSATUR        : " + countColours(d) + "  valid = " +
isValid(d));
        System.out.println("\nTimetable using DSATUR with room allocation:");
        allocateRooms(d, rn, rc);
        System.out.println();
    }

    public static void main(String[] args) {
        // Test 1: example from the problem statement (C1..C4, S1..S4)
        List<int[]> small = new ArrayList<>();
        small.add(new int[]{0, 1});   // S1: C1, C2
```

```
        small.add(new int[]{1, 2});   // S2: C2, C3
        small.add(new int[]{0, 2});   // S3: C1, C3
        small.add(new int[]{3});      // S4: C4
        runAll("Test 1: Example input (4 courses, 4 students)", 4, small,
                new String[]{"R1", "R2", "R3"}, new int[]{100, 80, 50});

        // Test 2: 15 courses, 40 students, each enrolled in 2 random courses (fixed
seed)
        Random rnd = new Random(7);
        List<int[]> big = new ArrayList<>();
        for (int s = 0; s < 40; s++) {
            Set<Integer> pick = new LinkedHashSet<>();
            while (pick.size() < 2) pick.add(rnd.nextInt(15));
            int[] arr = new int[2];
            int k = 0;
            for (int c : pick) arr[k++] = c;
            big.add(arr);
        }
        runAll("Test 2: 15 courses, 40 students, 2 courses each", 15, big,
                new String[]{"R1", "R2", "R3", "R4", "R5", "R6"}, new int[]{12, 10, 8,
7, 6, 5});
    }
}
```

Output:

```
=== Test 1: Example input (4 courses, 4 students) ===
Courses (vertices): 4, Conflicts (edges): 3
  C1 (2 students) conflicts with: C2, C3
  C2 (2 students) conflicts with: C1, C3
  C3 (2 students) conflicts with: C1, C2
  C4 (1 students) conflicts with: none

Algorithm comparison (number of exam slots):
  Greedy        : 3  valid = true
  Welsh-Powell  : 3  valid = true
  DSATUR        : 3  valid = true

Timetable using DSATUR with room allocation:
Course   Time Slot  Students   Room
C1       Slot 1     2          R3
C4       Slot 1     1          R2
C2       Slot 2     2          R3
C3       Slot 3     2          R3

=== Test 2: 15 courses, 40 students, 2 courses each ===
Courses (vertices): 15, Conflicts (edges): 34
  C1 (9 students) conflicts with: C2, C3, C4, C5, C7, C10, C13
  C2 (8 students) conflicts with: C1, C4, C9, C10, C13, C15
  C3 (3 students) conflicts with: C1, C14, C15
  C4 (6 students) conflicts with: C1, C2, C5, C6, C7
  C5 (7 students) conflicts with: C1, C4, C7, C8, C11, C13
  C6 (3 students) conflicts with: C4, C10, C11
  C7 (7 students) conflicts with: C1, C4, C5, C12, C13, C15
```

```
  C8 (4 students) conflicts with: C5, C11, C15
  C9 (2 students) conflicts with: C2, C15
  C10 (6 students) conflicts with: C1, C2, C6, C11, C13
  C11 (6 students) conflicts with: C5, C6, C8, C10, C13
  C12 (1 students) conflicts with: C7
  C13 (8 students) conflicts with: C1, C2, C5, C7, C10, C11, C14
  C14 (3 students) conflicts with: C3, C13, C15
  C15 (7 students) conflicts with: C2, C3, C7, C8, C9, C14

Algorithm comparison (number of exam slots):
  Greedy        : 5  valid = true
  Welsh-Powell  : 4  valid = true
  DSATUR        : 4  valid = true

Timetable using DSATUR with room allocation:
Course   Time Slot  Students   Room
C1       Slot 1     9          R2
C15      Slot 1     7          R4
C11      Slot 1     6          R5
C12      Slot 1     1          R6
C13      Slot 2     8          R3
C4       Slot 2     6          R5
C8       Slot 2     4          R6
C3       Slot 2     3          R4
C9       Slot 2     2          R2
C2       Slot 3     8          R3
C5       Slot 3     7          R4
C6       Slot 3     3          R6
C14      Slot 3     3          R5
C7       Slot 4     7          R4
C10      Slot 4     6          R5
```

Conclusion:

The conflict graph was built with adjacency sets, and all three algorithms produced valid, conflict-free timetables. On the sample input from the assignment, every algorithm used 3 slots, with C4 sharing a slot with C1 as expected. On the larger test with 15 courses and 40 students, Greedy used 5 slots while Welsh-Powell and DSATUR needed only 4, which shows that ordering vertices by degree or saturation gives better colourings. After the slots were fixed, each exam was placed in the smallest free room that could hold its students, so no room hosts two exams in the same slot. Graph colouring is NP-hard in general, but the heuristics with adjacency lists give a practical, scalable and conflict-free timetable for universities.